Day 111 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 16, Day 7 reference

Watch: Why Deep Neural Networks (DNNs) Underperform Tree-Based Models on Tabular Data (DataMListic) | Why do tree based models still outperform deep learning on tabular data? (AI Paper Cast)

`day111_trees_vs_neural_nets.py`, segment by segment, in the order `main()` uses things. The six exercise parts (Days 105-110) are copied verbatim and were explained on their own days; this guide recaps them in one segment, then walks the new code: the MLP, the model wrappers and the experiments. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: W, b = a layer's weights and biases · Z = a layer's weighted sums · A = its outputs · M, V = Adam's averages · wd = weight decay

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.join(HERE, "..", "week-15-practical-nlp-pretrained"))
with contextlib.redirect_stdout(io.StringIO()):
    import day99_exercise as d99
TITANIC = os.path.join(HERE, "..", "week-03-intro-to-machine-learning", "titanic.csv")
FEATS = ["Pclass", "is_female", "Age", "SibSp", "Parch", "Fare"]
SPLITS = {"Titanic": 20, "Breast cancer": 20, "Diabetes": 20, "Digits": 5}
PERTURB_SPLITS = 5
NOISE = [0, 30, 100, 300]
SIZES = {"Breast cancer": [50, 100, 200, 398], "Digits": [50, 100, 200, 400, 1257]}
GRID = [(32, 1e-4), (32, 0.1), (128, 1e-4), (128, 0.1)]
XGB = dict(eta=0.1, max_depth=3, seed=0, nthread=1, tree_method="hist")
```

- `import ast` — the parser Python itself uses: here, to find each function in the exercise files.
- `SPLITS = {"Titanic": 20, ...}` — fewer splits for digits, the slowest problem.
- `GRID = [(32, 1e-4), ...]` — the MLP's choices as (hidden units, weight decay) pairs.

Segment 1 — The week's parts

The week's parts, verbatim (one banner per exercise file)

CODE

```
# from day105_exercise.py, verbatim: gini, entropy, best_split
# ============================================================================
def gini(counts):                                       # NEW: 1 - sum p_k^2
    n = counts.sum(-1, keepdims=True)
    p = counts / np.maximum(n, 1)
    return 1 - (p ** 2).sum(-1)
...  (Days 106 to 110 follow, each under its own banner)
```

- `# from dayNNN_exercise.py, verbatim: ...` — a banner names each file and the functions taken from it; a generator pasted them in, so nothing was retyped.
- `best_split, grow_random, cost_path, ...` — Day 105's split search, Day 107's tree, Day 106's pruning: the pruned tree is built from three days' parts.

Segment 2 — The neural network

MLP: forward pass and loss

MATH

$$A_l = ReLU(A_{l-1} W_l + b_l); \quad dL/dZ = (p - y)/n \text{ for log loss, } (Z - y)/n \text{ for squared error}$$

CODE

```
class MLP:
    def __init__(self, sizes, task, seed):
        rng = np.random.RandomState(seed)
        self.task = task
        self.W = [rng.randn(a, b) * np.sqrt(2.0 / a) for a, b in zip(sizes[:-1], sizes[1:])]
        self.b = [np.zeros(b) for b in sizes[1:]]
        self.rng = rng
    def forward(self, X):
        self.A = [X]
        for i, (W, b) in enumerate(zip(self.W, self.b)):
            Z = self.A[-1] @ W + b
            self.A.append(np.maximum(Z, 0) if i < len(self.W) - 1 else Z)
        return self.A[-1]
    def loss_grad(self, Z, y):
        n = len(Z)
        if self.task == "binary":
            return float(np.mean(np.logaddexp(0, Z[:, 0]) - y * Z[:, 0])), (0.5 * (1 + np.tanh(Z /
                2)) - y[:, None]) / n
        if self.task == "multi":
            Z = Z - Z.max(1, keepdims=True); P = np.exp(Z); P /= P.sum(1, keepdims=True)
            Y = np.eye(Z.shape[1])[y]
            return float(-np.mean(np.log(P[np.arange(n), y] + 1e-12))), (P - Y) / n
        return float(np.mean((Z[:, 0] - y) ** 2) / 2), (Z - y[:, None]) / n
```

- `rng.randn(a, b) * np.sqrt(2.0 / a)` — He initialisation: suited to ReLU, keeps the signal's size steady through layers.
- `np.maximum(Z, 0) if i < len(self.W) - 1 else Z` — ReLU on hidden layers; the last layer stays linear.
- `np.logaddexp(0, Z[:, 0]) - y * Z[:, 0]` — log loss from log-odds without overflow.
- `0.5 * (1 + np.tanh(Z / 2))` — the sigmoid in a form that cannot overflow (an earlier version warned on extreme values).

MLP: backpropagation

MATH

$$dW_l = A_{l-1}^T dZ_l; \quad dZ_{l-1} = (dZ_l W_l^T) \cdot [Z_{l-1} > 0]$$

CODE

```
def grads(self, X, y):
    loss, dZ = self.loss_grad(self.forward(X), y)
    gW, gb = [None] * len(self.W), [None] * len(self.W)
    for i in range(len(self.W) - 1, -1, -1):
        gW[i] = self.A[i].T @ dZ; gb[i] = dZ.sum(0)
        if i:
            dZ = (dZ @ self.W[i].T) * (self.A[i] > 0)
    return loss, gW, gb
```

- `for i in range(len(self.W) - 1, -1, -1):` — counts down: from the last layer to the first.
- `gW[i] = self.A[i].T @ dZ` — the layer's input, transposed, times the gradient at its output.

- `(dZ @ self.W[i].T) * (self.A[i] > 0)` — back through the weights, then through ReLU: zero where the unit was off.

MLP: fit, with Adam and early stopping

MATH

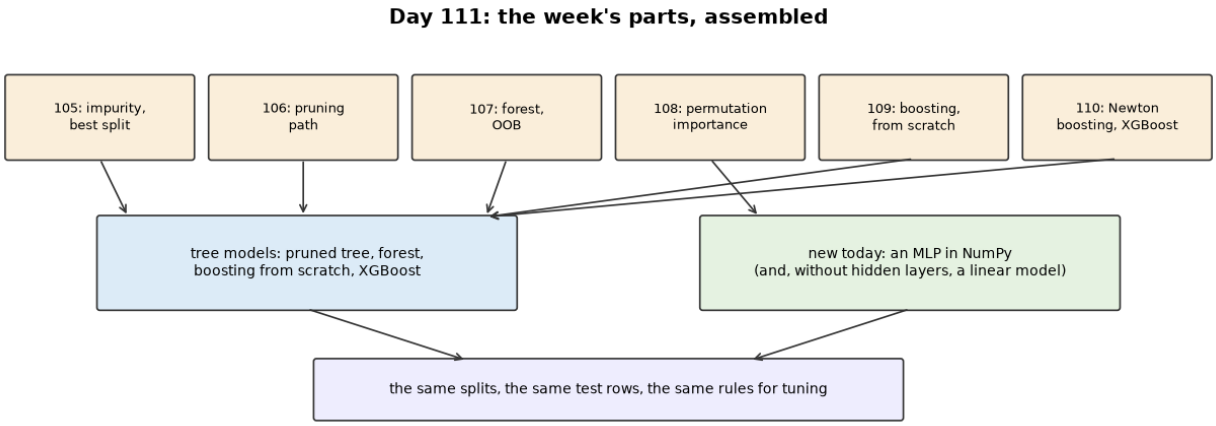
$m \leftarrow 0.9m + 0.1g$; $v \leftarrow 0.999v + 0.001g^2$; $p \leftarrow p - lr(m_hat/(\sqrt{v_hat} + \varepsilon) + wd \cdot p)$, with m_hat, v_hat the bias-corrected averages

CODE

```
def fit(self, X, y, Xv, yv, lr=1e-3, wd=1e-4, epochs=400, batch=64, patience=30):
    self.mu, self.sd = X.mean(0), X.std(0) + 1e-8
    if self.task == "regression":
        self.ym, self.ys = y.mean(), y.std()
        y, yv = (y - self.ym) / self.ys, (yv - self.ym) / self.ys
    X, Xv = (X - self.mu) / self.sd, (Xv - self.mu) / self.sd
    P = self.W + self.b; M = [np.zeros_like(p) for p in P]; V = [np.zeros_like(p) for p in P]; t =
        0
    best, best_p, wait = np.inf, None, 0
    for ep in range(epochs):
        order = self.rng.permutation(len(X))
        for s in range(0, len(X), batch):
            i = order[s:s + batch]; _, gW, gb = self.grads(X[i], y[i]); t += 1
            for k, (p, g) in enumerate(zip(P, gW + gb)):
                M[k] = 0.9 * M[k] + 0.1 * g; V[k] = 0.999 * V[k] + 0.001 * g * g
                step = (M[k] / (1 - 0.9 ** t)) / (np.sqrt(V[k] / (1 - 0.999 ** t)) + 1e-8)
                p -= lr * (step + (wd * p if k < len(self.W) else 0))
        v = self.loss_grad(self.forward(Xv), yv)[0]
        if v < best - 1e-6:
            best, best_p, wait = v, [p.copy() for p in P], 0
        else:
            wait += 1
            if wait >= patience:
                break
    for p, q in zip(P, best_p):
        p[...] = q
    self.epochs, self.best = ep + 1, best
    return self
```

- `P = self.W + self.b` — list concatenation: one list of every parameter array, so one loop updates them all.
- `p -= lr * (...)` — -= changes the array in place, so self.W and self.b see the update.
- `best_p = [p.copy() for p in P]` — a snapshot of the best weights; .copy() so later updates do not change it.
- `p[...] = q` — writes into the existing array: restores the best weights in place.

SYSTEM DIAGRAM



The week's parts, assembled.

Segment 3 — The models

The models as functions

CODE

```
def fit_mlp(X, y, seed, hidden=True):
    task = task_of(y); out = int(y.max()) + 1 if task == "multi" else 1
    a, v = carve(len(y), seed); best = None
    for h, wd in (GRID if hidden else [(0, 1e-4), (0, 0.1)]):
        sizes = [X.shape[1], h, h, out] if hidden else [X.shape[1], out]
        m = MLP(sizes, task, seed).fit(X[a], y[a], X[v], y[v], wd=wd)
        if best is None or m.best < best.best:
            best = m
    return best.predict
...
MODELS = {"pruned tree": fit_pruned, "random forest": fit_forest_oob, "boosting (scratch)":
    fit_boost_es, "XGBoost": fit_xgb_es,
          "linear model": lambda X, y, s: fit_mlp(X, y, s, hidden=False), "MLP": fit_mlp}
TREES = ["pruned tree", "random forest", "boosting (scratch)", "XGBoost"]
```

- `best = m if ... m.best < best.best` — keeps the setting with the lowest validation loss.
- `return best.predict` — returns a method: a function the caller can apply to test rows.
- `MODELS = {name: function}` — a dictionary of fit functions, so every experiment loops over the same models.

Segment 4 — The experiments

demo_checks: verbatim, by reading the exercise files

CODE

```
me = open(os.path.abspath(__file__), encoding="utf-8").read(); found = total = 0
for d in range(105, 111):
    src = open(os.path.join(HERE, f"day{d}_exercise.py"), encoding="utf-8").read()
    for node in ast.parse(src).body:
        if isinstance(node, ast.FunctionDef) and node.name != "node":
            total += 1; found += "\n".join(src.split("\n")[node.lineno - 1:node.end_lineno]) in me
```

- `ast.parse(src).body` — the top-level statements of a file, as objects.
- `node.lineno, node.end_lineno` — the first and last line of a function.

- `found += ... in me` — True counts as 1: the function's text is in this file or not.

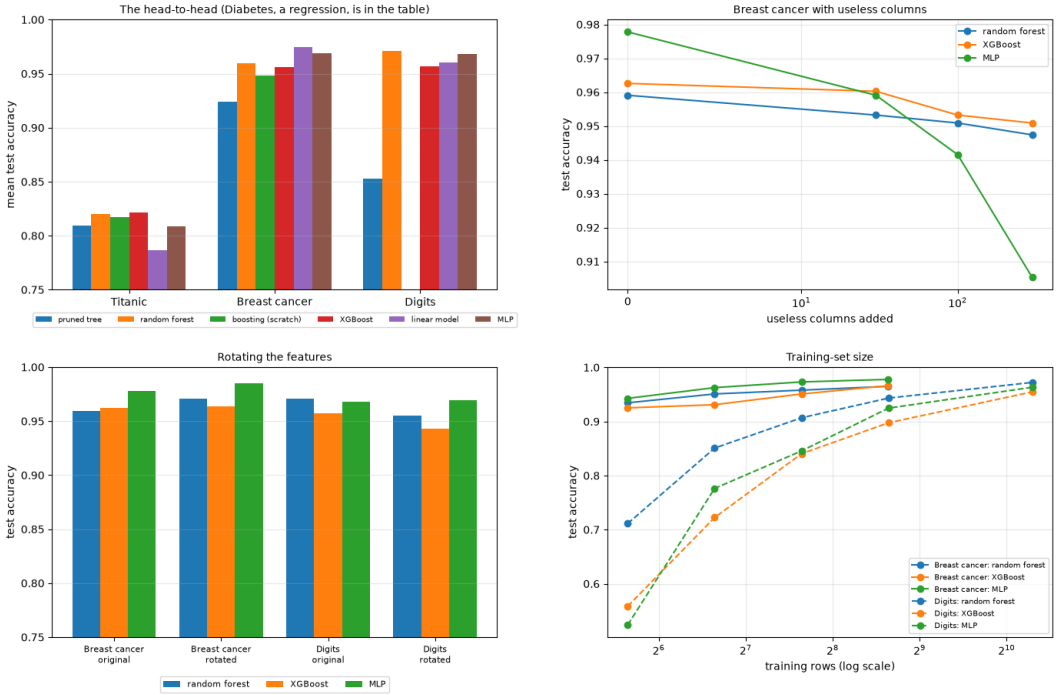
perturbed, rotation and the monotone warp

CODE

```
def perturbed(make, X, y, label, models=("random forest", "XGBoost", "MLP")):
    out = {}
    for m in models:
        v = []
        for s in range(PERTURB_SPLITS):
            tr, te = split(y, s); A, B = make(X[tr], X[te], s)
            v.append(score(MODELS[m](A, y[tr], s)(B), y[te]))
        out[m] = float(np.mean(v))
    print(f"  {label:<34}" + "".join(f"{out[m]:>15.4f}" for m in models))
    return out
def demo_useless(X, y):
    print("\n" + "=" * 70)
    print("3. USELESS COLUMNS -- breast cancer with random columns added")
    print("=" * 70)
    print(f"{PERTURB_SPLITS} splits; the columns are random numbers with no relation to the
        label")
    print(f"  {'':<34}{'random forest':>15}{'XGBoost':>15}{'MLP':>15}")
    rows = []
    for k in NOISE:
        add = lambda A, B, s, k=k: (np.hstack([A, np.random.RandomState(1000 +
            s).normal(size=(len(A), k))]),
                                    np.hstack([B, np.random.RandomState(2000 +
                                        s).normal(size=(len(B), k))]))
        rows.append((k, perturbed(add, X, y, f"{k} useless columns ({X.shape[1] + k} in all)")))
    print("Actual output from this lesson's run.")
    return rows
def rotation(p, seed):
    q, r = np.linalg.qr(np.random.RandomState(seed).normal(size=(p, p)))
    return q * np.sign(np.diag(r))
```

- `A, B = make(X[tr], X[te], s)` — make is a function that changes the table; it sees training rows to fit anything it needs.
- `np.linalg.qr(np.random.RandomState(seed).normal(size=(p, p)))` — QR of a random matrix gives a random orthogonal one.
- `q * np.sign(np.diag(r))` — a sign fix, so every orientation is equally likely.

REAL GRAPH



Real graph from the script: the head-to-head, useless columns, rotation, and training-set size.

Quick Syntax Glossary — new or reinforced constructs

- `ast.parse(src).body` — a file's top-level statements.
- `range(n - 1, -1, -1)` — count down to 0.
- `arr[...] = other` — assign into an existing array.
- `np.linalg.qr(A)` — the QR decomposition.
- `return obj.method` — return a bound method as a function.
- `lambda A, b, f=f: score(f(A), b)` — a default argument freezes f's current value.

End of Day 111 syntax reference. For the experiments and what they showed, see the Day 111 complete guide.